

Reporte Post-Laboratorio 09:

Estructuras de Datos Nativas e Iteradores en Python

Felipe Colli Olea

28 de mayo de 2026

1. Introducción

El presente informe documenta el desarrollo del Laboratorio 09, enfocado en el manejo de colecciones de datos mediante el uso de listas nativas de Python y la implementación de ciclos iterativos definidos (`for loops`).

Dejar atrás el procesamiento de variables escalares aisladas para manipular arreglos dinámicos representa un avance fundamental en la capacidad de cómputo. El laboratorio exigió aplicar estos conceptos a dos problemas clásicos de la ingeniería: el ajuste de datos a un modelo de regresión lineal (Mínimos Cuadrados Ordinarios) y el procesamiento condicional en bloque de un sistema de calificaciones académicas.

2. Desarrollo y Lógica Implementada

El laboratorio requirió el uso de listas vacías (`[]`) y el método de mutación `.append()` para construir arreglos dinámicamente:

- **Modelo de Mínimos Cuadrados:** Se recibieron dos listas crudas (altura en pulgadas y peso en libras). Primero, se iteró sobre ellas para generar nuevas listas (x e y) con los valores normalizados al sistema métrico. Luego, mediante un ciclo `for i in range(n)`, se emuló un recorrido por índices espaciales para acumular las sumatorias necesarias ($\sum X$, $\sum Y$, $\sum X^2$, $\sum XY$). Finalmente, se calcularon los parámetros y se iteró nuevamente para generar la lista de predicciones Y_2 .
- **Procesamiento de Calificaciones:** Se solicitó el ingreso de 15 notas. Se implementó un ciclo para solicitar *inputs* consecutivos al usuario. Posteriormente, se recorrió la colección utilizando un flujo condicional para bonificar cada calificación (multiplicando por 1,25), insertando un tope lógico de 7,0 para evitar el desbordamiento de la escala de evaluación, y finalizando con el cálculo de los promedios poblacionales.

3. Reflexiones Arquitectónicas y Dificultades

Dado mi perfil orientado a la Programación Competitiva en `C++`, la transición hacia la sintaxis y el manejo de memoria subyacente de Python generó fricciones conceptuales importantes durante el desarrollo de la actividad, revelando las falencias arquitectónicas del lenguaje interpretado:

1. **Gestión de Memoria y Punteros Implícitos:** En C++, la asignación de estructuras (como `std::vector`) realiza copias profundas por defecto (*deep copy*). En Python, el operador de asignación (`=`) sobre listas no duplica la información, sino que simplemente transfiere la referencia de memoria (puntero). Esta abstracción excesiva resulta peligrosa, ya que modificar una variable secundaria corrompe la data original de forma silenciosa, requiriendo el uso de librerías extra o rebanados (*slicing* `[:]`) para forzar copias reales.
2. **Parseo de Entradas (I/O Streams):** La función `input()` de Python captura el bloque completo como una cadena de texto. Para ingresar múltiples datos en una sola línea (un estándar de eficiencia en cualquier algoritmo), Python exige utilizar métodos de cadena (`.split()`) seguidos de mapeos a flotantes (`map()`). Esto contrasta negativamente con la elegancia del flujo de entrada de C++ (`std::cin >>`), el cual ignora automáticamente delimitadores invisibles y asigna los tipos de forma nativa e instantánea.
3. **Definición de *Scopes* y Tipado Estático:** Estructurar bucles `for` utilizando `range()` y delimitando el bloque funcional exclusivamente mediante indentaciones visuales carece de la rigurosidad y seguridad espacial que otorgan las llaves `{}` de C. Asimismo, la imposibilidad de declarar explícitamente el tipo de dato que albergará una lista (como `std::vector<int>`) quita previsibilidad al código y delega en el intérprete resoluciones que impactan severamente en el rendimiento.

4. Conclusiones

El Laboratorio 09 permitió consolidar el uso de iteradores y estructuras de datos dinámicas en Python. Lograr calcular la pendiente de una regresión lineal manipulando los datos desde su estructura cruda hasta las listas predictivas demostró la viabilidad del lenguaje para ejecutar modelamientos lógicos sin depender de módulos estadísticos externos.

Sin embargo, desde la óptica de la Ingeniería de Software, la experiencia reafirma que el alto nivel de abstracción de Python conlleva un costo en la seguridad del código. El tipado dinámico, la asignación implícita por referencias y el manejo poco estricto de las entradas por consola, son características que facilitan el *scripting* rápido, pero que resultan ser un retroceso metodológico para quien acostumbra a tener un control absoluto sobre el procesamiento asintótico y espacial de la memoria en lenguajes compilados.